

Module 10: Routing & Navigation

Building Multi-Page Angular Applications

ANGULAR

FRONTEND DEVELOPMENT

Module Overview

By the end of this module, you will be able to:

01

Understand Angular Router

Learn how the router maps URLs to components

02

Configure Routes & Parameters

Set up routes, route params, and query params

03

Build Nested Navigation

Organize parent-child routing hierarchies

04

Navigate Programmatically

Trigger navigation from business logic

05

Lazy Load & Scope Providers

Optimize bundles and manage feature-level services

Why Routing?

Modern applications serve multiple views — all without ever reloading the browser. Angular Router powers this seamless experience.

Common Pages

- **Login**
- **Dashboard**
- **Employees**
- **Orders**
- **Reports**
- **Settings**

SPA Benefits

- **Fast Navigation** — No full page reloads
- **Better UX** — Smooth, app-like transitions
- **Bookmarkable URLs** — Shareable deep links
- **Browser History** — Back/forward button support

Angular Router Fundamentals

The Angular Router intercepts URL changes and renders the matching component — keeping the app alive in a single browser page.



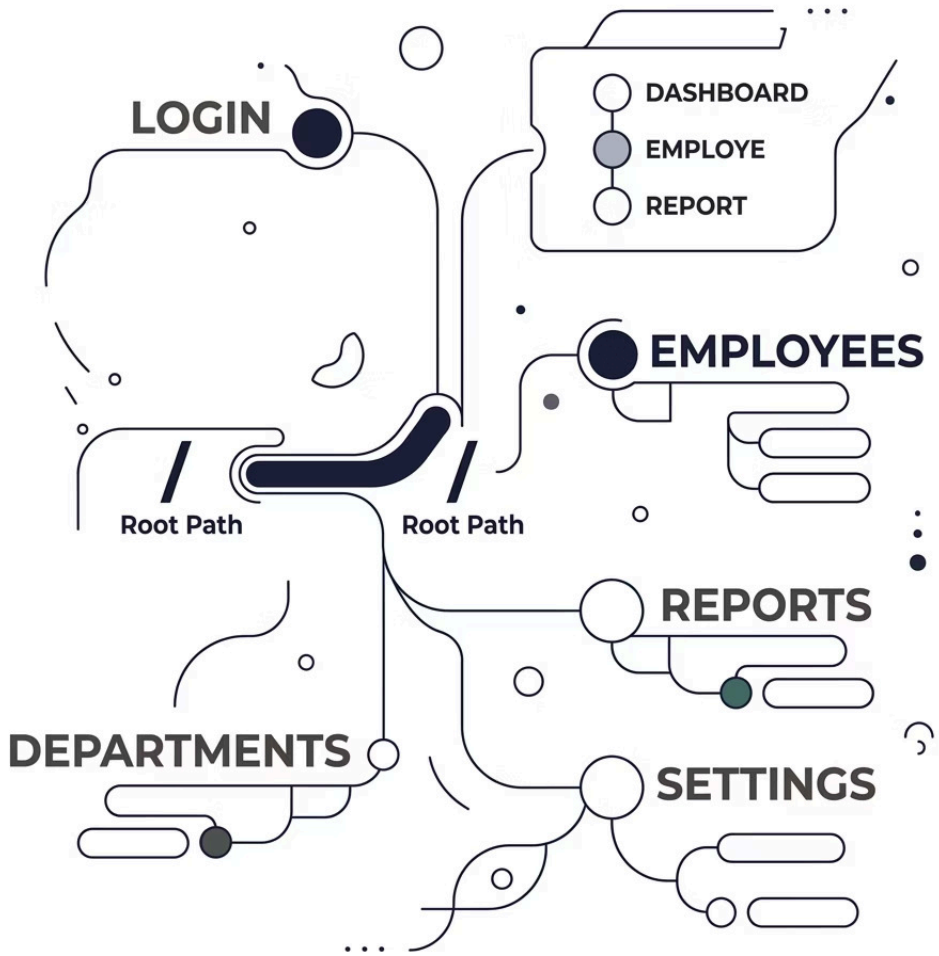
The router acts as the bridge between your URL and your component tree. It also manages route guards, lazy loading, and parameter extraction — making it the backbone of any multi-view Angular app.

Route Configuration

Routes are defined as a centralized array that maps URL paths to components. This gives your application a predictable, organized navigation structure.

- Organized by feature area
- Centralized route management
- Supports easy application expansion
- Enforces consistent URL conventions

Define routes once in your app config or a dedicated routing module and import them at the root level.



Route Parameters

Route parameters embed dynamic values directly in the URL path to identify specific resources — enabling clean, RESTful navigation.

Example URLs

```
/employees/101
/orders/5001
/products/250
```

Common Use Cases

- Employee & Customer detail pages
- Product detail views
- Order tracking pages



Dynamic Navigation

Navigate to any resource using its unique ID in the path.



Resource Identification

Uniquely identify records in a clean, readable format.



SEO-Friendly URLs

Descriptive paths improve discoverability and shareability.

Query Parameters

Query parameters append optional metadata to a URL without altering the route itself — perfect for filters, search, and pagination.

Example URLs

```
/employees?page=2
/products?category=laptop
/orders?status=completed
```

Common Use Cases

Search	**Sorting**
Filtering	**Pagination**
Language	**Theme**

Why Query Params?

- **Flexible** — Combine multiple params freely
- **Shareable** — Full state is preserved in the URL
- **Non-destructive** — Route stays the same
- **Easy to read** — Human-readable key-value pairs

Nested & Child Routes

Large applications group related views using parent-child routing — enabling shared layouts and modular navigation hierarchies.



Why Nested Routes?

- **Better organization** — Group related views together
- **Shared layouts** — Parent renders common UI (sidebars, headers)
- **Reusable navigation** — Tab bars and sub-menus per feature
- **Modular design** — Each feature owns its own route subtree

Use `router-outlet` inside parent components to render child routes in place.



Programmatic Navigation

Not all navigation is triggered by a link click. Angular's `Router` service lets you navigate from anywhere in your code — driven by business logic and application events.

Auth Events

Redirect to dashboard on login, or back to login on logout.

Form Submissions

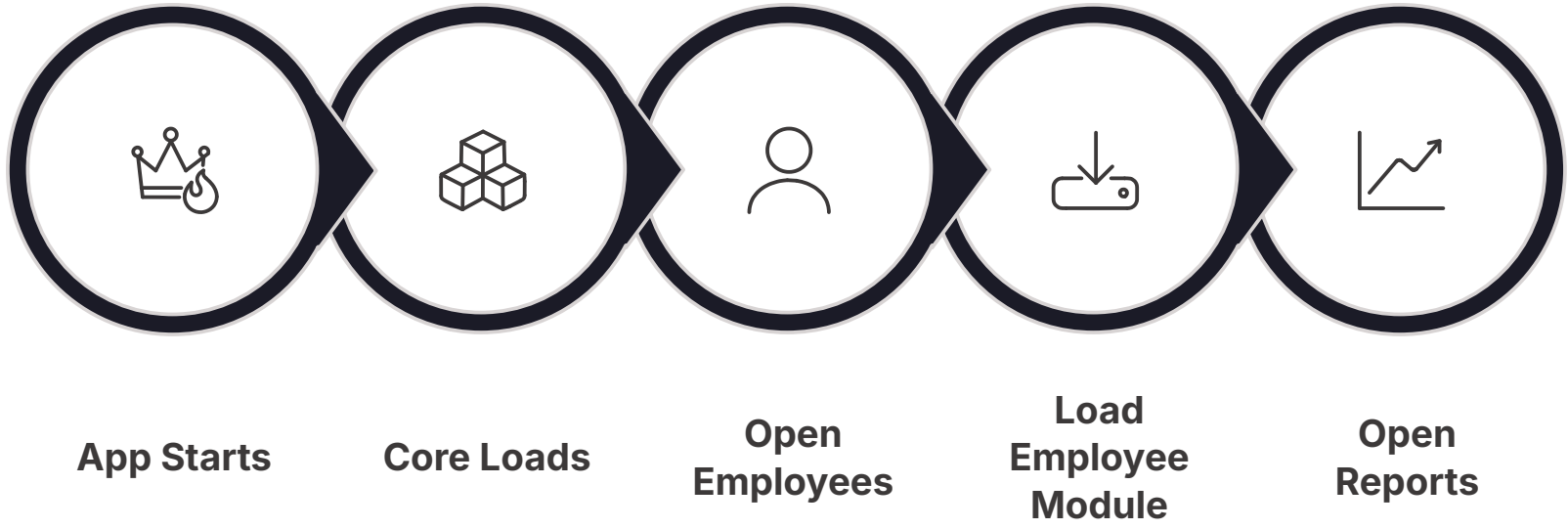
Navigate to a detail view after saving a record successfully.

Access Control

Redirect unauthorized users to an error or login page.

Lazy Loading

Lazy loading defers loading feature modules until a user actually navigates to them — dramatically reducing the initial bundle size.



Performance Benefits

- Faster initial startup time
- Smaller first-load bundle
- Better scalability as app grows

Best Candidates to Lazy Load

Employees	**Reports**
Admin	**Settings**

Route-Level Providers

Angular allows services to be scoped directly to a route — meaning they are created when the route is activated and destroyed when the user leaves.

Advantages

- **Feature-specific** — Only available where needed
- **Better encapsulation** — Logic stays inside the feature
- **Reduced memory usage** — Cleaned up on route exit
- **Improved modularity** — Features are self-contained

Common Use Cases

Feature Services

Scoped data or state for one feature module

Route Configuration

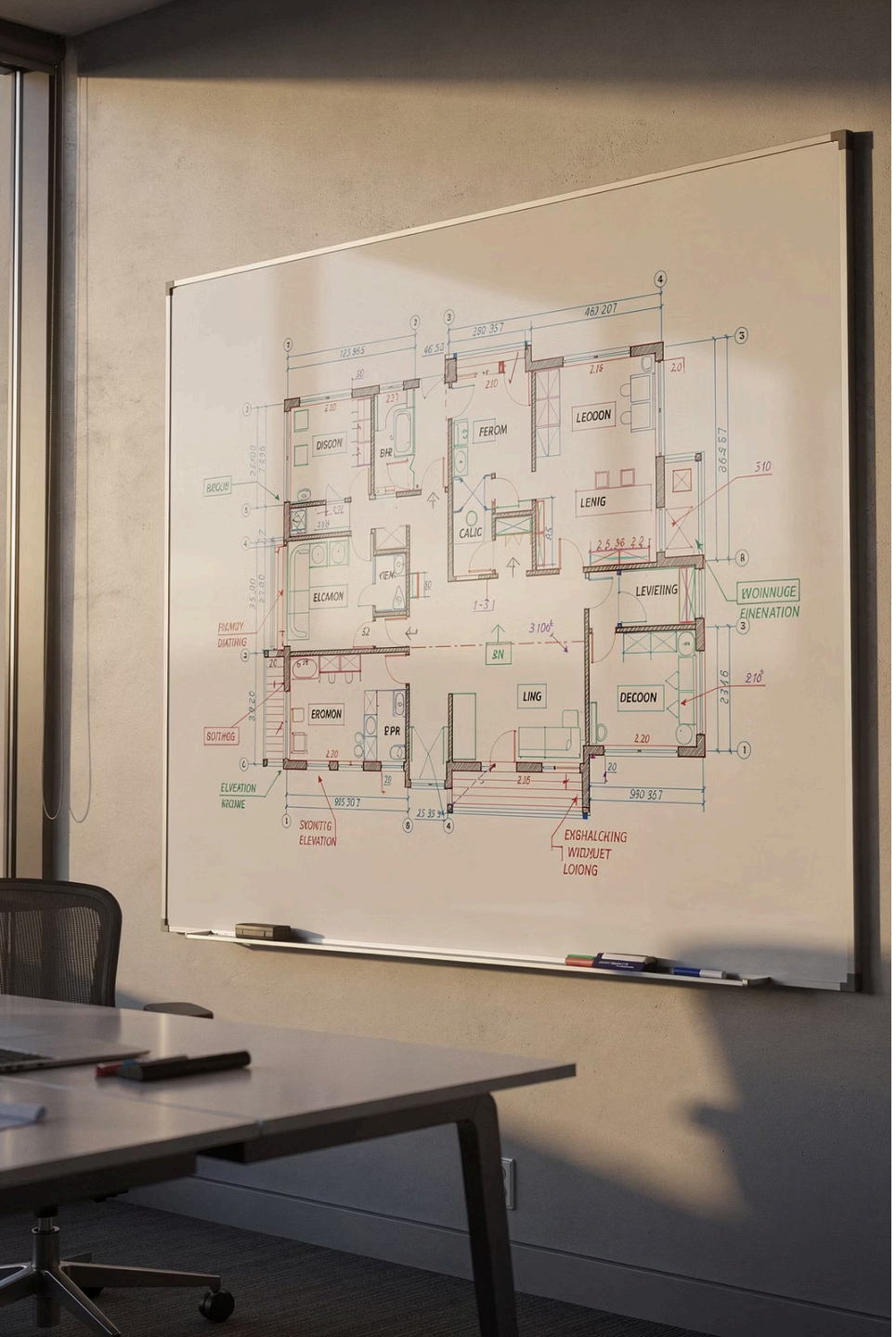
Inject settings specific to a single route

Temporary State

State that should reset on navigation away

Isolated Logic

Business logic isolated from the global scope



Routing Best Practices

Organize routes by feature

Group related routes together and keep each feature's routing self-contained.

Keep URL structure meaningful

Use route parameters for resource IDs; use query params for filters, search, and sorting.

Lazy load large feature areas

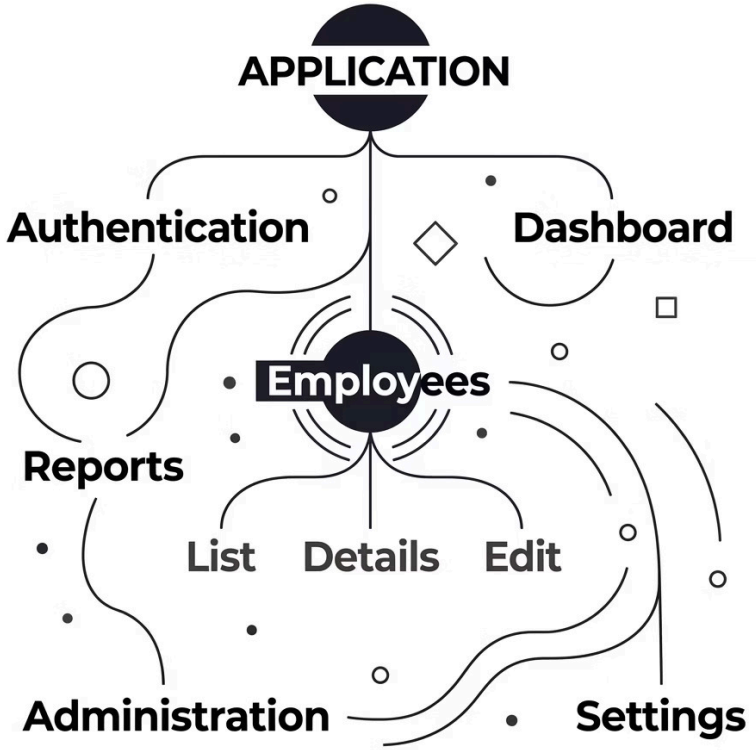
Defer anything that isn't needed on first load — Employees, Reports, Admin, Settings.

Scope providers appropriately

Prefer route-level providers for feature-specific services over global singleton injection.

Enterprise Routing Architecture

A well-structured enterprise Angular app organizes routes around feature areas — enabling independent development, clear hierarchy, and easy maintenance.



Enterprise Benefits

- **Feature-based routing** — Each team owns its route subtree
- **Clear navigation hierarchy** — Predictable URL conventions
- **Independent development** — Features can be built in parallel
- **Easy maintenance** — Isolated changes with minimal side effects

Apply lazy loading at each top-level feature route for maximum scalability.

Module Summary

What We Covered

- **Angular Router Fundamentals**
- **Route Configuration**
- **Route & Query Parameters**
- **Nested & Child Routes**
- **Programmatic Navigation**
- **Lazy Loading**
- **Route-Level Providers**
- **Routing Best Practices**

Module 11: Route Guards & Data Resolution

- Route Guards — protecting navigation
- Authentication & Authorization flows
- Route Resolvers — pre-loading data
- Data Prefetching strategies